University of Cambridge

Department of Physics

LoRA-Tuned Large Language Models for Time-Series Forecasting of Dynamical Systems

M2 Coursework Report

Author: Saam Nazempour (sn665)

Supervisor: Dr. Cranmer

March 29, 2025

Word Count: 2926

Abstract

This study explores the use of a LoRA-fine-tuned Qwen2.5-Instruct language model for time-series forecasting of predator-prey dynamics governed by the Lotka–Volterra equations. Leveraging the LLMTIME framework, we convert numerical trajectories into tokenized text sequences, enabling autoregressive prediction via language modeling. We systematically investigate the effects of learning rate, LoRA rank, and context length on forecasting accuracy and computational cost. Our results show that larger ranks and higher learning rates accelerate convergence, while longer context windows improve long-range modeling at the expense of increased FLOPs. The final model—trained with a learning rate of 0.0001, LoRA rank 8, and context length 256—achieves strong predictive performance with a median R^2 above 0.85 across ten systems. This work highlights the potential of parameter-efficient LLM adaptation for structured numerical forecasting under constrained compute budgets.

Contents

1 Introduction **3**

2 Theory **4**

2.1 Transformers 4

2.2 Qwen Architecture 5

2.3 LoRA Fine-Tuning 5

3 Implementation **7**

3.1 Preprocessing 7

3.2 Forecasting 7

3.3 Training the LoRA-augmented Qwen Model 7

3.4 FLOPs Estimation 8

4 Results **9**

4.1 Untrained Qwen Model Performance 9

4.2 Hyperparameter Tuning Experiments 10

4.3 Final Model Performance 13

5 Conclusion **16**

A Appendix **18**

1. Introduction

Recent work by Gruver et al. has shown that Large Language Models (LLMs) can perform time-series forecasting—even without explicit pretraining on numerical data [1]. This surprising capability has lead to much deeper investigation into how LLMs, originally built for natural language tasks, can be adapted to structured time series and has been a focus of recent research [2] [3] [4].

In this project, we explore the use of LoRA (Low-Rank Adaptation) to fine-tune Qwen2.5-Instruct—a 0.5B parameter instruction-tuned LLM—for forecasting predator-prey dynamics governed by the Lotka–Volterra equations [5]. The dataset comprises 1,000 independent 2-variable time series (prey and predator) of 100 time steps each, offering a well-defined domain for learning temporal patterns.

We apply the LLMTIME preprocessing framework to convert multivariate numerical sequences into text, enabling seamless integration with Qwen's tokenizer and generation interface. Forecasting is cast as a generative task: the model is prompted with past population values and asked to generate future values in the same format, which are then parsed into numeric predictions.

The paper begins with a theoretical overview of the Qwen model, tokenization process, and LoRA methodology. We then detail our implementation pipeline—including preprocessing, training, forecasting, and FLOPs estimation. Finally, we present results comparing the base (untrained) model with LoRA-tuned variants, analyzing the impact of hyperparameters such as learning rate, rank, and context length. We conclude with practical recommendations for fine-tuning LLMs on numerical tasks under tight computational budgets.

2. Theory

2.1 Transformers

Transformers are a class of neural networks originally introduced for sequence modeling tasks such as translation, and are now widely used in large language models (LLMs). The core component of the Transformer is the self-attention mechanism, which enables the model to learn contextual relationships between elements of a sequence—whether they are adjacent or far apart.

In self-attention, each token in the input sequence is first projected into three learned vectors: a query vector $\vec{q}$, a key vector $\vec{k}$, and a value vector $\vec{v}$. These vectors are then used to compute how much attention each token should pay to the others in the sequence [6]. The output of self-attention is computed as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{2.1}$$

Here, $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are the matrices of queries, keys, and values for a sequence of length T, and d_k is the dimensionality of the key vectors. The dot product $\mathbf{Q}\mathbf{K}^T$ computes a score between all pairs of tokens, which is scaled and passed through a softmax to obtain attention weights. These weights are then used to produce a weighted sum of the value vectors. A graphic depicitng the inner working of self-attention blocks is shown in Fig. 2.1 [7].

Transformer layers typically use multi-head attention, which replicates this process in parallel multiple times with separate learned projections, allowing the model to attend to different aspects of the sequence simultaneously.

In decoder-only Transformers like Qwen2.5, the model generates output tokens autoregressively, meaning it predicts the next token using only the past tokens as context. This makes them naturally suited to generation tasks—whether of text or, in our case, forecasting future points in a time series.

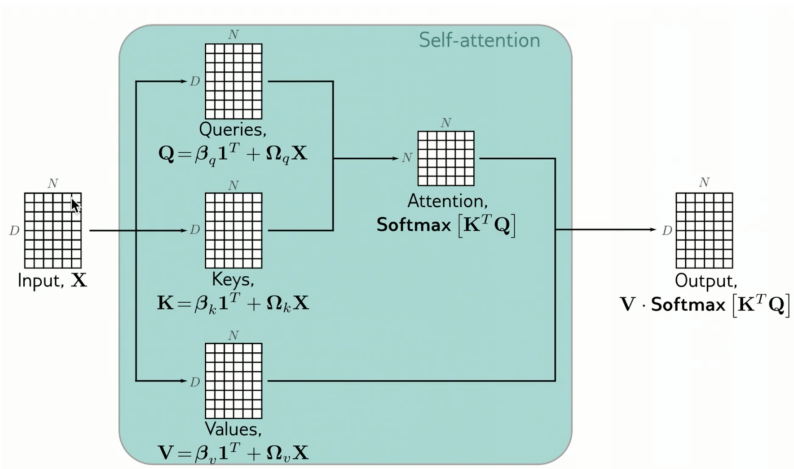

Figure 2.1: Transformer self-attention mechanism. Each token attends to every other token in the sequence via scaled dot-product attention using learned query, key, and value projections. [7]

2.2 Qwen Architecture

Qwen2.5 is a family of open-source decoder-only language models released by Alibaba Cloud [8]. It follows the architectural lineage of modern Transformer-based models such as LLaMA and GPT, while introducing several tailored enhancements for efficiency and downstream adaptability. For this coursework, we fine-tune the instruction-tuned version of Qwen2.5-0.5B-Instruct, which consists of approximately 500 million parameters and is specifically designed for lightweight deployment and instruction following.

Qwen2.5 incorporates several modern enhancements to the Transformer decoder architecture:

1. **Group Query Attention (GQA):** A variant of multi-head attention which reduces memory and compute cost by sharing keys and values across multiple query heads.
2. **SwiGLU activation function:** Replaces traditional activation functions like ReLU or GELU, improving training stability and performance.
3. **Rotary positional embeddings (RoPE):** Encodes token positions directly into the attention mechanism, allowing better extrapolation to longer sequences.
4. **Tokenization via tiktoken:** The model uses a tokenizer trained with BPE (Byte-Pair Encoding) and is compatible with the GPT tokenizer ecosystem.

In our project, we fine-tune the Qwen2.5 model using a Low-Rank Adaptation (LoRA) approach (described in Section 2.3), allowing us to efficiently adapt the model to time series forecasting tasks using limited computational resources.

2.3 LoRA Fine-Tuning

Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning technique introduced by Hu et al. (2021) to reduce the computational and memory burden of adapting large language models to downstream tasks [9]. Instead of updating all model parameters during training, LoRA introduces small, trainable low-rank matrices into specific parts of the model (typically the attention layers), allowing the base model weights to remain frozen.

The key idea is that, during fine-tuning, the weight update matrix often has low intrinsic rank. LoRA explicitly models this by decomposing the weight update into a product of two smaller matrices:

$$\Delta W = BA \tag{2.2}$$

where $A \in R^{r \times d}$ and $B \in R^{r \times d}$ are trainable parameters, and $r \ll d$ is the rank of the adaptation. The updated weight used at inference time becomes:

$$W' = W_0 + \alpha BA \tag{2.3}$$

where W_0 is the frozen pre-trained weight and α is a scaling factor (often set such that α/r normalizes the adaptation).

In practice, LoRA is typically applied to the query and value projection matrices in the attention mechanism. For example, if W_q is the original query projection matrix, it is replaced with:

$$W_q(x) = W_q^{(0)}x + \frac{\alpha}{r}BAx \tag{2.4}$$

Here, W_q^0 remains fixed during training, and only A and B are updated.

LoRA introduces a small number of trainable parameters into specific parts of a frozen model, enabling efficient fine-tuning with minimal memory and compute overhead. It preserves pre-trained knowledge, avoids catastrophic forgetting, and supports modularity by allowing task-specific adapters to be swapped in without retraining the base model. In our setup, we apply

LoRA to the q_proj and v_proj components of each self-attention layer in Qwen2.5-Instruct. This configuration enables effective adaptation for time series forecasting under tight computational budgets.

3. Implementation

3.1 Preprocessing

We preprocess the Lotka–Volterra predator-prey datasets using the LLMTIME framework, which encodes time series as text sequences. Each sequence consists of comma-separated prey and predator pairs, delimited by semicolons.

The model is provided with the first T time steps (e.g., 60) as context and tasked with autoregressively generating the next H time steps (e.g., 40). We tokenize the sequences using the Qwen2.5 tokenizer without special tokens or padding. To ensure compatibility with the model's context length, we apply truncation or sliding window segmentation where needed.

3.2 Forecasting

We perform autoregressive forecasting with Qwen2.5-Instruct—optionally fine-tuned using LoRA—using Hugging Face's generate API. The model is prompted with tokenized floating-point values for the first $T = 60$ time steps of a Lotka-Volterra trajectory and is expected to generate the continuation up to step 100.

Since each time step consists of two values (prey and predator), this requires generating at least 80 valid numeric outputs. However, due to the variability in token-to-number decoding, a shorter generation may yield fewer than 40 steps and cause shape mismatches during evaluation. To prevent this, we conservatively over-generate tokens to ensure that enough values are produced after decoding.

Greedy decoding is used to eliminate randomness: the most probable token is selected at each step. Once generation is complete, the output string is decoded, numeric values are extracted using regular expressions, and the first 80 values (40 time steps) are rescaled ($\times 10$) and interpreted as the predicted prey and predator trajectories.

3.3 Training the LoRA-augmented Qwen Model

To fine-tune the Qwen2.5-Instruct model on the Lotka-Volterra forecasting task, we adopted a parameter-efficient fine-tuning strategy using Low-Rank Adaptation (LoRA). Specifically, we inserted LoRA adapters into the query and value projection layers of each transformer block. These adapters introduce a pair of learnable low-rank matrices while freezing the original model weights, significantly reducing the number of trainable parameters.

The model was trained using teacher forcing on a masked language modeling objective, where the model is tasked with autoregressively predicting the next token in the sequence given the preceding context. Training was conducted exclusively on CPU using the Hugging Face transformers library and Accelerate for efficient gradient management and device-agnostic training.

We experimented with a range of LoRA ranks, learning rates, weight decays, and batch sizes to evaluate their impact on convergence and forecasting performance. Context length was also varied to study its effect on generalization. These ablations are reported in detail in Section 4.

Throughout training, both training and validation losses were recorded at regular intervals.

Model checkpoints were saved periodically, and the best-performing models were retained for evaluation and visualization.

3.4 FLOPs Estimation

To evaluate the computational efficiency of our LoRA fine-tuning experiments, we analytically estimated the number of floating point operations (FLOPs) required for training and inference. This estimation provides insight into the cost-performance trade-offs associated with different hyperparameter configurations (e.g. LoRA rank, context length, number of steps).

We decompose the transformer architecture into its fundamental operations, and for each layer, we compute FLOPs associated with:

1. **Multi-Head Self-Attention (MHA):** This includes projections for queries, keys, and values, attention score computation, and output projection.
2. **MLP Block:** Using a SwiGLU activation, this includes two matrix multiplications and an element-wise activation.
3. **LayerNorm:** Two per layer, each with cost linear in the hidden size.
4. **LoRA Adapter Paths:** We explicitly add the FLOPs contribution of the LoRA modules injected into the query and value projections, computed as an additional pair of low-rank matrix multiplications.

For example, the FLOPs for a single transformer layer (with sequence length L, hidden size d, LoRA rank r, and number of heads h) are:

$$\begin{aligned}\text{FLOPs}_{\text{layer}} = \text{SelfAttention}(L, d, h) + \text{MLP}(L, d, d_{\text{ff}}) + 2 \cdot \text{LayerNorm}(L, d) \\ + \text{LoRA}(L, d, h, r)\end{aligned} \tag{3.1}$$

The total FLOPs for a forward pass through the full model is simply the sum over all layers. Backpropagation is estimated as approximately twice the cost of the forward pass, leading to the total training FLOPs:

$$\text{FLOPs}_{\text{train}} = 3 \times \text{FLOPs}_{\text{forward}} \times \text{steps} \tag{3.2}$$

Inference FLOPs are computed using the forward-only path, scaled by the number of evaluation sequences. The exact formulas used for each module were implemented in code, using standard cost formulas for dense matrix multiplication:

$$(m \times n) \mathbin{@} (n \times p) \Rightarrow m \cdot p \cdot (2n - 1) \tag{3.3}$$

All reported FLOP estimates in Section 4 are derived using these analytical expressions applied to our actual training hyperparameters.

4. Results

4.1 Untrained Qwen Model Performance

Forecasting

To establish a baseline, we evaluate the untrained Qwen2.5 model's ability to perform zero-shot forecasting on the Lotka-Volterra predator-prey dynamics. The model is prompted with the first 60 time steps of a system's trajectory, formatted using the LLMTIME scheme, and tasked with autoregressively forecasting the remaining 40 time steps (from step 60 to 100).

Fig. 4.1 illustrates the predictions of the untrained model across six distinct system IDs. Despite having no prior training, the model exhibits some sensitivity to the input signal and produces forecasts that occasionally align in phase or frequency with the ground truth. However, most predictions lack long-term structure and drift significantly over time — often diverging toward flat or monotonic dynamics that fail to capture the cyclical predator-prey oscillations. This highlights the necessity of fine-tuning, as the model on its own does not possess useful inductive biases for this class of dynamical systems.

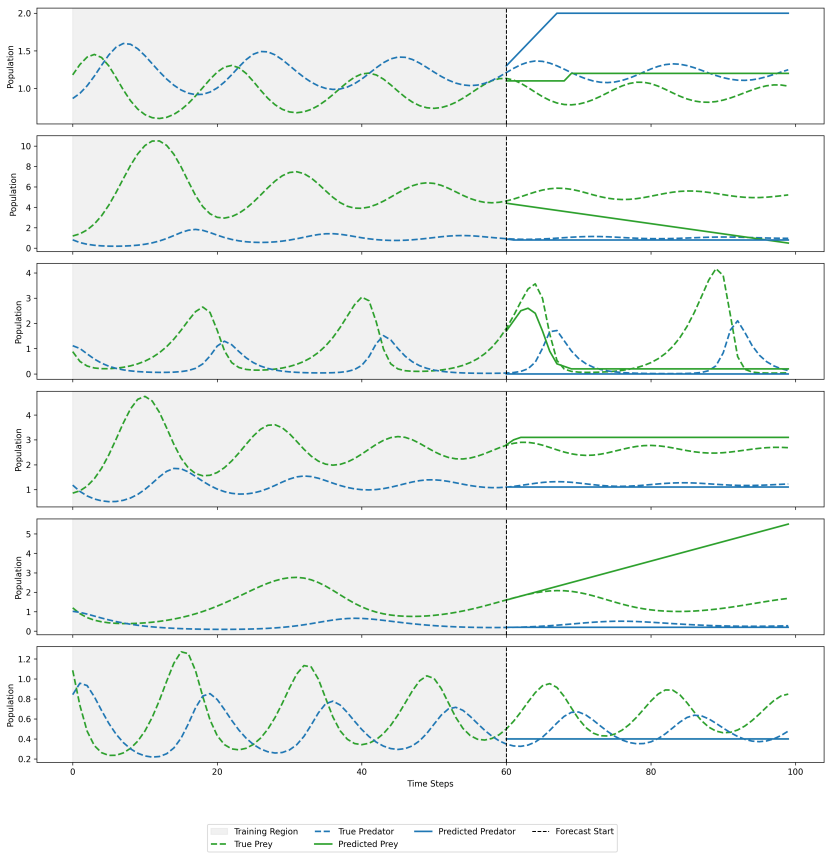

Figure 4.1: Forecasts of untrained Qwen2.5-Instruct across multiple Lotka–Volterra systems. While some trends are partially captured, overall prediction accuracy and stability is low.

Computational Cost

Generating the forecast sequence for each system required up to 1,000 new tokens due to the inefficiencies of decoding numerical sequences with an autoregressive LLM. This resulted in approximately $\mathbf{1.56 \times 10^{14}}$ floating point operations (FLOPs) per system, based on the Qwen architecture and LoRA configuration used in this coursework.

These results highlight the poor performance–compute trade-off of using large untrained LLMs for forecasting tasks. Without fine-tuning, the model lacks both accuracy and efficiency.

4.2 Hyperparameter Tuning Experiments

Learning Rate vs LoRA Rank

We begin our hyperparameter exploration by jointly varying the LoRA rank and learning rate, training nine models for 500 steps each with a fixed context length of 256 tokens. The training and validation loss curves are shown in Fig. 4.2. While all models converge to relatively low losses within 500 steps, their convergence speed and final performance vary substantially across configurations.

We observe that larger LoRA ranks and higher learning rates consistently accelerate convergence. Notably, models such as `lr5e-05_rank8` and `lr0.0001_rank8` achieve sub-0.2 validation loss in under 150 steps. In contrast, smaller ranks and lower learning rates generally converge more slowly and plateau at higher loss values.

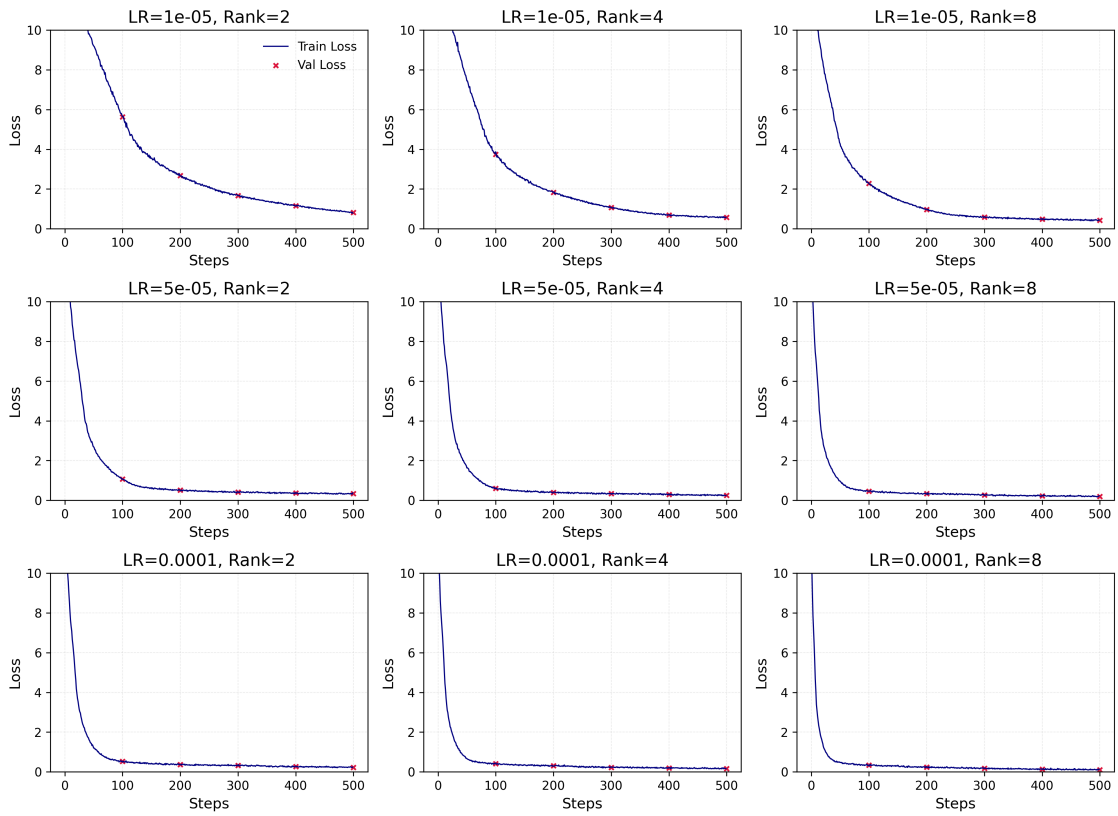

Figure 4.2: Training and validation loss against steps for all possible combinations of learning rate and LoRA rank. Although all configurations eventually converge, the higher the LoRA rank and learning rate, the faster the convergence speed.

To quantify the relationship between convergence dynamics and generalization, we compute the convergence step for each model using a smoothed validation loss curve. Specifically, we apply a moving average with a window size of 50 and define convergence as the first step at

which the relative improvement in smoothed loss drops below 0.01%. This conservative threshold captures the point where training ceases to yield meaningful loss reduction. We then plot the final validation loss against this convergence step in Figure 4.3. A clear negative correlation emerges: faster-converging models tend to achieve lower final loss. This trend suggests that, under fixed compute budgets, convergence speed can serve as a practical proxy for downstream forecasting performance.

Tab. 4.1 summarizes the validation losses, convergence steps, and estimated training FLOPs across all learning rate and LoRA rank combinations. The results reveal a clear trade-off between accuracy, convergence speed, and computational cost. Models with higher ranks and learning rates tend to converge faster but incur greater compute, while lower learning rates require more steps and FLOPs to reach optimal performance. Among all configurations, the model with a learning rate of 0.0001 and LoRA rank of 8 achieves the lowest validation loss (0.1083) and fastest convergence (101 steps), making it the most efficient overall. This model is therefore selected for final training and evaluation in Section 4.3.

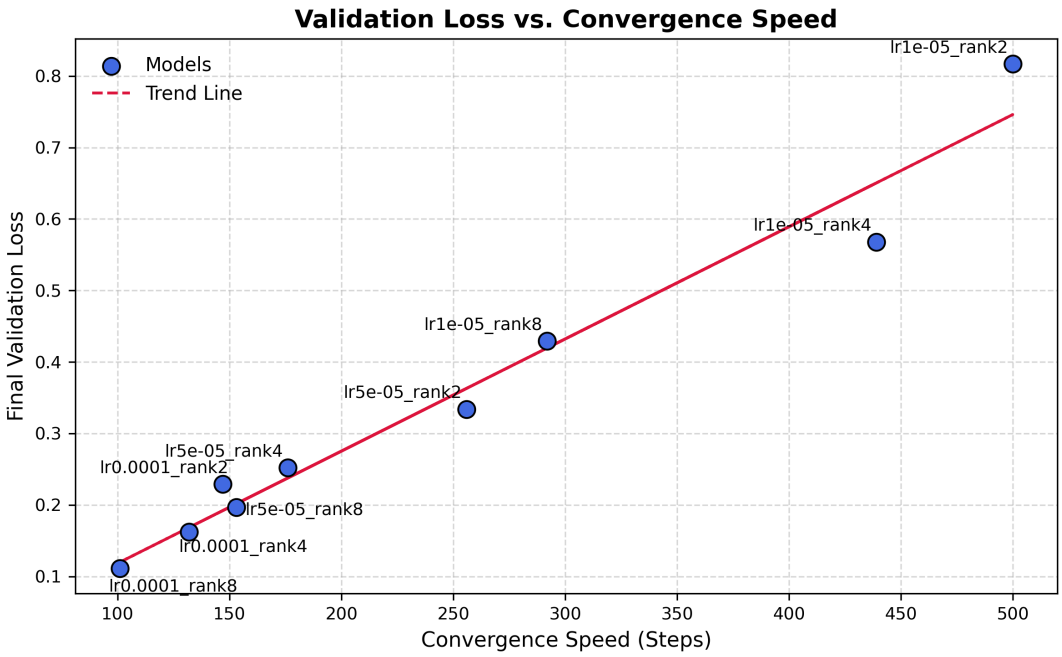

Figure 4.3: Final validation loss versus convergence step for different LoRA hyperparameter configurations.

Model	Validation Loss	Convergence Step	Training FLOPs ($\times 10^{14}$)
lr1e-5_rank2	0.817	500	29.4
lr1e-5_rank4	0.568	439	25.8
lr1e-5_rank8	0.429	292	17.2
lr5e-5_rank2	0.334	256	15.0
lr5e-5_rank4	0.252	176	10.3
lr5e-5_rank8	0.192	153	9.0
lr0.0001_rank2	0.229	147	8.6
lr0.0001_rank4	0.162	132	7.8
lr0.0001_rank8	**0.1083**	**101**	**5.9**

Table 4.1: Validation loss, convergence step, and estimated training FLOPs for each learning rate and LoRA rank configuration. Best model in bold.

Context Length Tuning

We now investigate the effect of context length on training dynamics and forecasting performance by training models with LoRA rank 8 and learning rate 0.0001 across three context lengths: 128, 512, and 768 tokens. Each model is trained for 400 steps, and the corresponding training and validation losses are shown in Figure 4.4.

Longer context lengths consistently lead to lower validation loss, reflecting improved model capacity to learn long-range temporal dependencies in the Lotka-Volterra dynamics. However, this gain comes at a significant computational cost. Training FLOPs increase substantially with context length due to the quadratic scaling of attention mechanisms. As shown in Tab. 4.2, moving from 128 to 768 tokens raises the training cost by more than 6× (from 0.90×10^{14} to 5.89×10^{14} FLOPs). Moreover, longer contexts increase per-step memory and runtime overhead, slowing down training iterations.

Despite these costs, the model trained with 768 tokens achieves the best average R^2 score across systems. However, diminishing returns are observed beyond 512 tokens in some cases. To balance computational efficiency with performance, we select a context length of 256 tokens for the final model — a midpoint that retains sufficient forecasting accuracy while remaining tractable under our compute constraints.

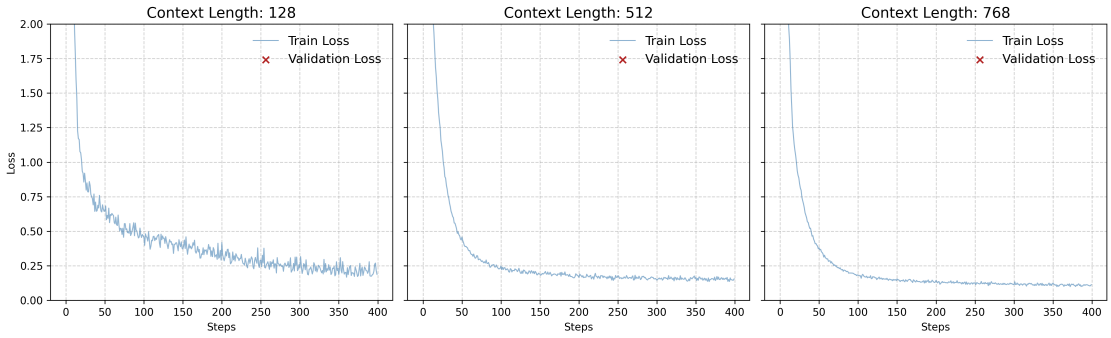

Figure 4.4: Training and validation loss curves for context lengths 128, 512, and 768. Longer contexts improve final loss but increase training cost, motivating a trade-off in context selection.

System	Context Length	MSE	R^2 Score	Training FLOPs ($\times 10^{14}$)
System 1	128	0.326	-3.72	0.90
	512	0.084	0.45	3.79
	768	0.122	-0.78	5.89
System 2	128	14.28	-2.15	0.90
	512	0.530	-0.88	3.79
	768	14.28	-2.15	5.89
System 3	128	5.14	-7.97	0.90
	512	6.69	-1.00	3.79
	768	15.82	-26.63	5.89
System 4	128	0.454	-0.78	0.90
	512	158.45	-74.10	3.79
	768	4.10	-0.94	5.89

Table 4.2: Forecasting performance and training FLOPs across four systems for models trained with context lengths of 128, 512, and 768 tokens. Longer contexts generally yield better accuracy, but incur greater compute cost.

4.3 Final Model Performance

Forecasts

Having identified the optimal configuration in Section 4.2 specifically a learning rate of 0.0001, LoRA rank 8, and context length of 256—we now evaluate this model's forecasting performance on 10 unseen predator-prey systems.

Fig. 4.5 presents the resulting predictions against ground truth trajectories. The model is able to consistently generate long-term forecasts with high fidelity, accurately capturing the cyclical dynamics and phase relationships between predator and prey populations across a wide variety of systems. The dashed lines represent ground truth values beyond the training window, while the solid lines show the model's extrapolated forecasts.

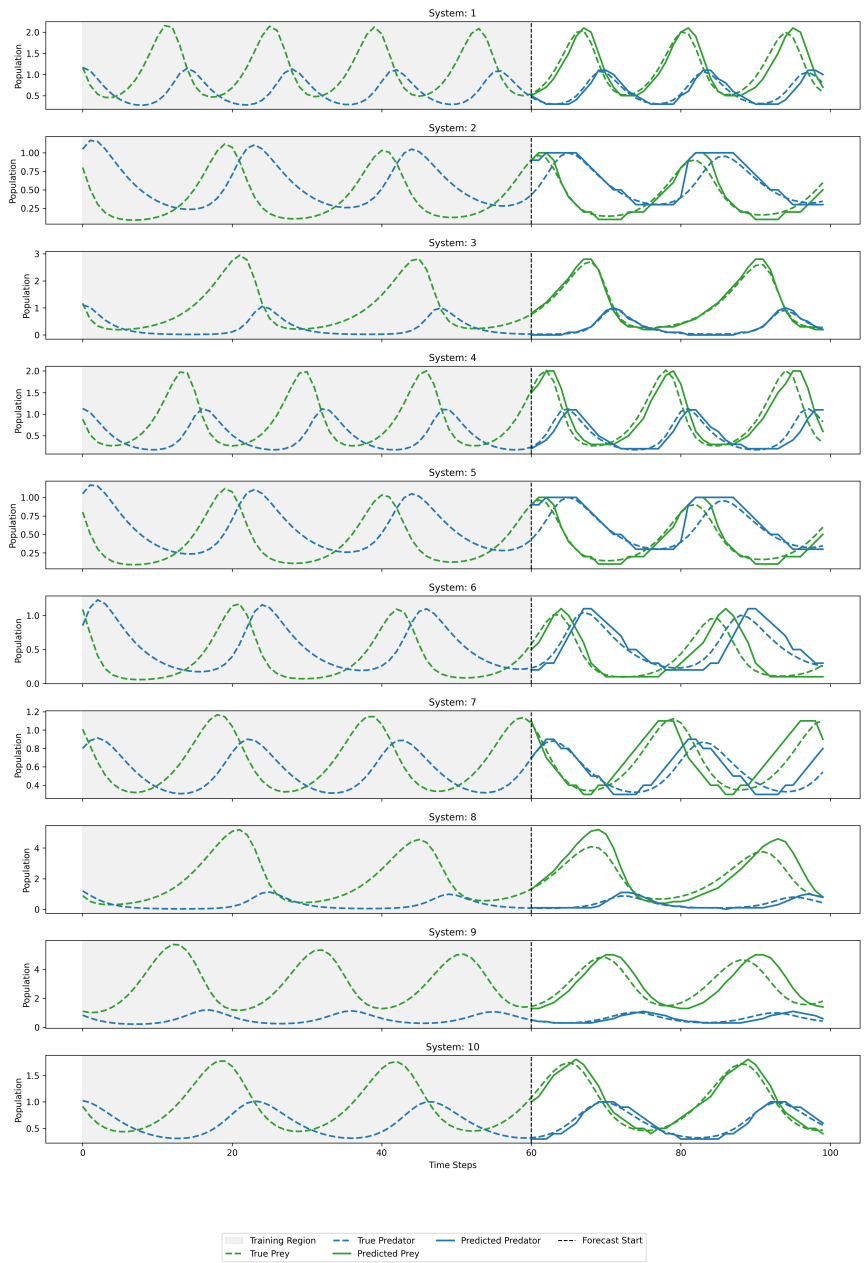

Figure 4.5: Forecasts of the final trained Qwen2.5-LoRA model on 10 evaluation systems. Forecasts begin at time step 60 and extend to 100. The model captures the cyclic prey-predator dynamics.

Computational Cost and Loss Curve

To better understand the final model's learning behaviour, we plot the training and validation loss over the full 15,000 training steps (Fig. 4.6). The training loss steadily declines and plateaus, while the validation loss remains low and stable across evaluation checkpoints, indicating no signs of overfitting.

The total training cost for the final model configuration (context length 256, LoRA rank 8, 24 layers) is estimated at $\mathbf{6.86 \times 10^{16}}$ FLOPs. This reflects the full cost of backpropagation across 15,000 steps and provides a benchmark for future compute-aware fine-tuning strategies.

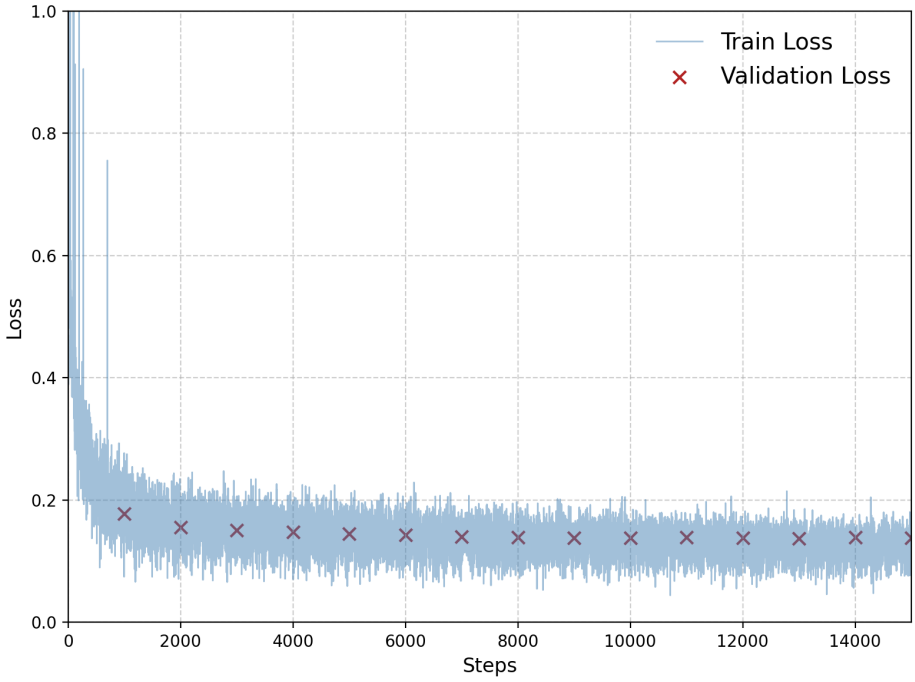

Figure 4.6: Training and validation loss curve for the final model across 15,000 steps.

Quantitative Performance Metrics

To quantitatively assess performance, we compute a range of metrics across the forecast horizon (time steps 60–100). These include Mean Squared Error (MSE), Mean Absolute Error (MAE), Coefficient of Determination (R^2), Median Absolute Error (MedAE), Dynamic Time Warping (DTW), and Pearson correlation.

System	MSE	MAE	R^2 Score	MedAE	DTW	Pearson
1	0.023936	0.115251	0.903528	0.087113	0.893156	0.956013
2	0.014284	0.071380	0.793386	0.044355	0.903033	0.942218
3	0.005249	0.056619	0.999060	0.040131	0.637583	0.997974
4	0.043518	0.150483	0.851242	0.115788	1.048611	0.927711
5	0.014284	0.071380	0.793386	0.044355	0.903033	0.942218
6	0.020776	0.110883	0.770441	0.089703	0.600591	0.940178
7	0.013536	0.091715	0.765534	0.071264	0.681897	0.891696
8	0.278208	0.338082	0.811387	0.218586	3.333431	0.955676
9	0.288848	0.369850	0.855647	0.191678	2.006477	0.979731
10	0.005494	0.060681	0.967776	0.054831	0.620349	0.985357

Table 4.3: Forecasting performance of the final trained model on 10 systems. The model achieves high accuracy and strong correlation across multiple metrics.

As shown in Tab. 4.3, the final model consistently achieves low MSE and MAE across all systems, with a median R^2 score above 0.85. The lowest R^2 (0.76) still represents a strong fit, while several systems exceed 0.95. Additionally, DTW and Pearson correlation indicate strong alignment in shape and phase, confirming that the model captures not only values but also temporal dynamics accurately.

These results demonstrate that with targeted fine-tuning via LoRA and careful hyperparameter selection, large language models can learn and generalize the structure of dynamical systems from compact input representations.

5. Conclusion

This study applied a LoRA-fine-tuned Qwen2.5 model to forecast Lotka-Volterra predator-prey dynamics. Starting from an untrained baseline, we showed that targeted fine-tuning significantly improves forecasting accuracy. Through systematic hyperparameter tuning, we found that learning rate, LoRA rank, and context length critically affect convergence speed, performance, and compute cost.

The final model—using a learning rate of 0.0001, rank 8, and context length 256—offered the best balance of accuracy and efficiency, achieving strong results across all forecasting metrics. These findings highlight the potential of parameter-efficient adaptation methods for scientific time-series forecasting.

Bibliography

[1] N. Gruver, M. Finzi, S. Qiu, and A. G. Wilson, "Large language models are zero-shot time series forecasters," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: https://github.com/ngruver/llmtime

[2] C. S. Chen, G. L. Oliveira, H. Sharifi-Noghabi, and T. Sylvain, "Llm-ts integrator: Integrating llm for enhanced time series modeling," *arXiv preprint arXiv:2410.16489*, 2024, under review. [Online]. Available: https://arxiv.org/abs/2410.16489

[3] M. L. Wolff, S. Yang, K. Torkkola, and M. W. Mahoney, "Using pre-trained llms for multivariate time series forecasting," *arXiv preprint arXiv:2501.06386*, 2025. [Online]. Available: https://arxiv.org/abs/2501.06386

[4] R. Cao and Q. Wang, "An evaluation of standard statistical models and llms on time series forecasting," *arXiv preprint arXiv:2408.04867*, 2024. [Online]. Available: https://arxiv.org/abs/2408.04867

[5] P. J. Wangersky, "Lotka-volterra population models," *Annual Review of Ecology and Systematics*, vol. 9, no. 1, pp. 189–218, Nov. 1978.

[6] Y. Hao, L. Dong, F. Wei, and K. Xu, "Self-attention attribution: Interpreting information interactions inside transformer," in *Proceedings of the Thirty-Fifth AAAI Conference on Artificial Intelligence (AAAI-21)*. Virtual Conference: AAAI Press, 2021, pp. 12 963–12 971. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/17533

[7] S. J. Prince, *Understanding Deep Learning*. MIT Press, 2025, draft version dated March 26, 2025. Available at http://udlbook.com. [Online]. Available: http://udlbook.com

[8] Qwen Team, "Qwen2.5 Technical Report," Qwen Team, Tech. Rep., 2025, version 2, January 6, 2025. [Online]. Available: https://github.com/QwenLM/Qwen2.5

[9] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," *arXiv preprint arXiv:2106.09685*, 2021. [Online]. Available: https://arxiv.org/abs/2106.09685

A. Appendix

I used generation tools supportively for the following purpose:

Grammar checkers (Quillbot):

I first wrote the entire paper and checked it through manually. Then, I put each paragraph, one at a time, into QuillBot's 'Grammar Checker' functionality to check for potential grammar/spelling mistakes or to see if I could improve fluency with sentence restructuring.